ContractGuard · 合同风险审查官

技术文档 · 2026 第九届传智杯全国大学生数智创新与 AI 应用大赛
AI WEB 网页开发挑战赛 · 本科 B 组

项目定位	面向个人的合同风险提示工具（非法律意见）
技术形态	Vue 3 前端 + FastAPI 服务端，LLM / RAG / 多模态
覆盖范围	房屋租赁、实习劳动、培训贷预付卡 三类合同，24 项审查
文档版本	v1.0 · 2026-11-10

目 录

一、项目概述 3

二、需求分析 4

三、系统架构设计 5

四、AI 技术方案 7

五、核心功能实现 11

六、测试与部署 13

七、总结与展望 15

附录 A：AI 技术选型说明 16

附录 B：创新点说明 17

说明：本文档中的性能数据、命中率均为本机实测（Windows 11 / Python 3.13 / Node 22）， 测试集为公开合同示范文本，具体口径在第四、六章注明。

一、项目概述

1.1 背景

租房、实习、报培训班，是多数人最早接触到的三类"专业合同"。这类合同普遍存在两个问题： 一是条文由对方提供，格式条款居多；二是签署人缺少法律训练，无法判断哪一条会让自己吃亏。 常见的坑集中在押金退还、违约金比例、单方解除、转租限制、试用期与竞业限制等少数几个点位上， 但这些点位分散在不同法规中，普通人既不知道去哪查，也不知道查到的条文与眼前这句话是什么关系。

现有工具大多面向企业法务，做的是批量审查与风险台账；通用大模型则倾向于给出"总体感觉"， 缺少可核对的依据。本项目试图填这两者之间的空档：面向个人、只做三类合同、每条结论都能落到具体法条。

1.2 目标用户

用户	典型场景	核心诉求
应届生 / 实习生	签三方协议、实习协议、劳动合同	试用期是否合规、违约金能不能拒、竞业限制有没有补偿
租房青年	第一次独立租房、合租、转租	押金怎么退、涨租怎么办、维修谁负责
培训消费者	报名培训班、被推荐分期贷款	能不能退费、贷款是谁放的、违约金是否合理

1.3 产品定位与边界

产品定位是风险提示工具，不是法律服务。这一定位直接约束了输出的形态：

- 只指出"这一条有风险、依据是哪条法、可以往哪个方向改"，不输出可直接粘贴进合同的完整条文；
- 不预测诉讼结果、不评价胜诉可能性；
- 覆盖三类合同，其余类型明确拒绝，不做泛化；
- 不做多轮法律问答，避免滑向"AI 律师"的合规灰区。

> 这条边界不是自我限制，而是产品能在"有依据"和"不越界"之间站住的前提。 一旦输出完整条款，性质就从"提示"变成了"代拟文书"。

1.4 交付形态

Web 应用，前后端分离：前端负责合同阅读、结论呈现与交互；服务端负责解析、检索、模型编排与数据存储。 演示环境支持单机部署（单进程 uvicorn + 静态前端），不依赖外部数据库与向量服务。

二、需求分析

2.1 功能需求

编号	功能	优先级	说明
F1	合同上传与解析	P0	文字版 PDF 与合同照片（JPG/PNG），同步解析，超时转轮询
F2	条款级切分	P0	按"第X条"切分，保留页码、坐标与全文偏移
F3	风险审查 Agent	P0	规划 → 预筛 → 检索 → 判定 → 汇总，SSE 流式推送
F4	条款与结论对照	P0	双栏联动，结论可定位到原文条款
F5	法条依据溯源	P0	每条结论绑定法条编号与原文引用，可展开核对
F6	修改方向与沟通建议	P0	给出方向性描述与一句可用的沟通话术
F7	安全评分	P1	扣分制，规则固定，可复现
F8	法规资料库	P1	按法规浏览、关键词筛选、语义检索
F9	低置信度兜底	P1	置信度 < 0.6 折叠为"待复核"，不计入评分
F10	报告导出	P1	一键打印为 PDF：安全分、分级统计、逐条结论、依据与话术

2.2 非功能需求

类别	要求
性能	首页首屏 < 1.5s；30 页 PDF 解析 < 10s；单份合同审查 < 40s
安全	按文件头（magic number）校验类型；文件重命名为 UUID；路径穿越防护；原文件接口只按 ID 查库
隐私	匿名会话隔离；日志只记录长度与哈希，不记录合同正文；文件可随时删除
合规	全站声明"不构成法律意见"；每条结论附免责提示
可复现	评分由代码计算；检索参数、提示词版本、清单版本全部落库

2.3 范围取舍

在有限时间内，我们主动砍掉了四类需求，理由是它们不产生差异化、却会显著增加风险：

- 通用合同审查：覆盖面越广，单项准确率越低，且无法给出稳定的评测口径；
- 多轮法律问答：交互复杂度高，且极易越界为法律咨询；
- 企业批量审查：目标用户不是企业，功能堆叠无收益；
- 用户体系：MVP 阶段用匿名会话，避免引入账号与权限的完整链路。

三、系统架构设计

3.1 总体结构



3.2 关键设计决策

决策	选择	理由
条款切分粒度	按"第X条"，款级编号不另起条款	合同的最小语义单位是"条"；把（一）（二）切成独立条款会让上下文断裂，实测一份 15 页合同会从 15 条膨胀到 80 条
检索融合方式	RRF（k=60）	BM25 与余弦相似度量纲不同，分数加权会被长尾压死；RRF 只依赖名次
向量检索实现	numpy + npz	语料仅数千条，矩阵乘即可；避开向量库同步客户端在 async 路由中阻塞事件循环的问题
SSE 传输	POST + fetch 流式读取	原生 EventSource 只支持 GET，无法携带请求体
评分算法	代码计算，模型不参与	保证可复现、可解释、可调参
图片解析	多模态模型直读	一次调用完成识别与结构化；避开 OCR 的安装成本与"只出文字不出语义"的问题

3.3 数据模型

核心五张表，关系为 文档 → 条款 → 结论，审查记录挂在文档下：

```
documents(id, session_id, filename, stored_name, file_path, mime_type,
          sha256, contract_type, type_source, page_count, pages_json, status)
clauses(id, document_id→documents, seq, clause_no, page_no, bbox_json,
        text, char_start, char_end)
reviews(id, document_id→documents, status, safety_score, summary, model,
        prompt_version, checklist_version, latency_ms, started_at, finished_at)
findings(id, review_id→reviews, clause_id→clauses, item_id, risk_level,
         category, title, reason, legal_basis_json, suggestion,
         negotiation_tip, confidence)
knowledge_docs(id, source, title, article_no, article_index, doc_order,
               version, effective_date, content, checksum)
```

两个实现层面的补充：`clauses.bbox_json` 统一存 PDF 点坐标（左上原点、绝对坐标），前端用 pdf.js 的 `convertToViewportRectangle` 换算；`knowledge_docs.article_index` 存条号整数，因为字符串排序会把"第十条"排在"第二条"前面。

3.4 一次审查的数据流

```
上传 → 按 mime 分流
        ├─ PDF  → PyMuPDF 抽文本行（页码 + bbox）→ 条款切分 → 类型识别（关键词打分）
        └─ 图片 → 多模态模型读图 → 结构化条款（无坐标）→ 类型识别（模型判断兜底）
     → 落 documents / clauses
     → POST /documents/{id}/review（SSE）
        plan   加载清单（按合同类型）
        gate   条款 × 审查项 候选匹配（关键词 + BM25）
        judge  逐项检索法条 → 构造 prompt → JSON 判定（3 并发）
        agg    去重、排序、安全分、摘要
     → 落 reviews / findings，事件逐条推给前端
     → 前端按 finding 渲染卡片、按 clause 高亮原文
```

四、AI 技术方案

4.1 条款级切分

切分不是简单的正则匹配。真实合同里存在三个问题：标题与正文被排成两行、正文引用其他条文 （"依据本法第五百一十条"）、以及无编号段落。我们的处理顺序是：

1. 按行抽取文本，每行带页码、bbox、全文偏移；
2. 识别行首编号（第四级优先级：第X条 → 第X款 → 阿拉伯多级 → 括号编号）；
3. 若全文存在"第X条"，则只按条切分，款级编号保留在条内文本中；
4. 全文无编号时，按句末标点与 600 字上限兜底切块；
5. bbox 取条款起始页所有行的并集。

> 第 3 步是踩过坑之后改的。最初按文档规范把款也当切分点，结果"第一条 房屋基本情况"被下面的 （一）（二）（三）切成四条，一份 15 页合同切出 80 条，且每条 bbox 只剩一小段。 改成只按条切之后是 15 条 + 1 条前置说明，高亮框也能框住整条。

4.2 法规语料与索引

项目	数值
语料来源	民法典（2020 通过）、劳动合同法（2012 修正）、民事诉讼法（2023 修正）、 消费者权益保护法（2013 修正）、社会保险法（2018 修正）
入库条数	1260 + 98 + 306 + 63 + 98 = 1825 条，三类合同 24 个审查项引用的法条全部有原文
索引 chunk 数	1841（超长条文按 800 字 + 100 字重叠再切）
语料体积	约 620 KB（随代码入库，离线可跑）

抓取环节有一个容易忽略的问题：法条正文里会出现"依照本法第五百一十条"这类引用， 如果按正则全局匹配，它会被当成下一条的起点，把真正的条文截断。 实现时先筛出行首的编号，再在相邻两个行首编号之间取值。 民事诉讼法取的是人大网发布的 PDF 原件，正文按版面断行，需要按句读重新拼行后再切分。

4.3 检索：BM25 + 向量 + RRF

```
query（条款原文 + 审查项 query）
  ├─ BM25 召回 top 20（jieba 分词，cut_for_search）
  ├─ 向量召回 top 20（L2 归一化后点积；未配置 embedding 时自动跳过）
  ├─ RRF 融合（score = Σ 1/(k + rank)，k = 60）
  └─ 截断 top 5 → 拼进提示词
```

未配置 embedding 时只走 BM25，索引 `meta.json` 的 `dim` 记为 0， 检索层自动降级，不影响可用性。这是刻意的：演示环境不需要为了向量而依赖外部服务。

检索质量实测（人工构造 11 条典型查询，看 Top3 是否命中目标条文； 命中按"法规名 + 条号"判定，因为不同法的条号会撞）：

查询	目标条文	结果
租赁期限届满 返还租赁物	民法典 733	命中
试用期 期限 六个月 上限	劳动合同法 19	命中
竞业限制 期限 二年 补偿	劳动合同法 23/24	命中
转租 未经同意 解除合同	民法典 716	命中
违约金 过高 调整	民法典 585	命中
解除合同 不可抗力	民法典 563	命中
格式条款 排除限制消费者权利	消费者权益保护法 26	命中
消费者 自主选择 商品服务	消费者权益保护法 9	命中
用人单位 三十日 社会保险登记	社会保险法 58	命中（Top2）
协议管辖 书面 约定 人民法院	民事诉讼法 35	未命中（Top1 是涉外编 277）

10 项计分查询命中 9 项。已知未命中两项：「押金 退还 期限」——民法典并未直接规定押金， 召回的是"建设用地使用权期限"，属语义层面的固有缺口，靠清单的 query 设计规避（见 4.4）； 「协议管辖」召回的是民诉法涉外编第二百七十七条，字面几乎一样，但租赁合同该用的是第三十五条。

参考法条通道：不靠检索猜

后一类错误给出了一个明确信号：纯相似度排序在"同一部法里两处用语高度相似"时必然出错。 因此判定前先走一条确定性通道——清单每个审查项都人工标注了 `legal_refs` （如"民事诉讼法 第三十五条"），按"法规名 + 条号"在索引里精确取条，置顶塞进提示词， 再让 BM25 结果补位。清单是人工校过的，比检索猜得准；检索仍然保留，用来覆盖清单没列出、 但条款中确实涉及的周边条文。

实测效果：同一条租赁合同，「争议解决方式未选定」的结论依据从"待复核"变为 "民事诉讼法 第三十五条"，且引用的法条原文与结论语义一致。

4.4 条款预筛：控制上下文的成本命门

如果对每个审查项都把整份合同塞进模型，30 页合同的 token 与耗时都不可接受。 预筛的作用是把"条款 × 审查项"的笛卡尔积收敛到几十个相关组合：

- 以审查项的 `keywords` 命中数为主信号（合同用词比法条更口语，关键词更稳）；
- 以该文档内部的 BM25 分数为兜底（覆盖同义表述）；

- 每项最多取 5 条候选，候选为空则标记为"不涉及"，不调用模型。

实测一份 15 页合同（16 条条款）→ 10 个审查项全部有候选，进入 10 次判定； 一份 3 条的图片合同 → 只匹配到 4 处，其余 6 项直接判定"不涉及"，审查耗时从 25 秒降到 3.7 秒。

> 预筛还暴露了一个产品问题：清单里的 `query` 不能直接拿审查项名称。 例如"押金退还"在民法典里没有对应表述，必须改写成"租赁期限届满 承租人应当返还租赁物"才能召回第 733 条。 三类合同 24 项的 query 全部按法条原话重写过，这是准确率的一部分。

4.5 Agent 工作流

```
plan   按合同类型加载清单（version 随结果落库，便于复现）
gate   条款预筛，产出 (审查项, 候选条款) 列表
judge  逐项：检索法条 → 构造提示词 → JSON 判定 → 校验
agg    同条款同类别去重（保留更严重者）→ 安全分 → 摘要
save   写 reviews / findings，clause_seq 映射为数据库 id
```

并发控制：判定阶段 3 并发，单条超时 25 秒，单条失败不中断整体，标记 `confidence = 0` 并在摘要中提示"需人工复核"。

全流程以异步生成器形式产出事件（step / finding / progress / done / error）， 由 SSE 逐条推给前端，因此用户看到的是"结论一条条长出来"，而不是等待 10 秒后一次性刷新。

4.6 防幻觉：四条硬约束

法务场景里"看起来对"是最危险的。我们在四个层面做了约束：

层面	机制	效果
提示词	legal_basis 的 chunk_id 必须来自提供的法条列表；无检索结果时 legal_basis 必须为空且 confidence ≤ 0.5；不得输出确定性结论	约束模型"不许编"
后校验	逐条比对返回的 chunk_id 与本次检索结果，不在集合内的引用直接丢弃	即使模型编造，也进不了结论
置信度压制	如果 legal_basis 为空，confidence 强制压到 0.5 以下；< 0.6 在界面上折叠为"待复核"，且不计入评分	无依据的结论不会显得可信
失败兜底	JSON 解析失败重试一次；仍失败则返回 hit=false、confidence=0，并在摘要提示	宁可漏判，不可胡说

这套约束的代价要说清楚：确实会有结论落到"待复核"，需要人工对照原文看一眼。 我们接受这个代价——宁可显示一条保守的结论，也不显示一条编造的依据。 语料补全之后这类情况已经很少：五部法入库后，三类合同的 24 个审查项都有可引用的原文。

4.7 多模态：图片合同

租房场景大量是拍照合同，只支持 PDF 会丢掉一大半真实输入。我们比较了两种方案：

方案	实现成本	缺陷
PaddleOCR	Windows + Python 3.13 安装成功率低，包体接近 1GB	只输出文字，仍需自己判断哪句是"第一条"
多模态模型直读	一次 API 调用，无本地依赖	没有坐标，无法画高亮框

选择后者。提示词要求模型输出 `{"contract_type", "clauses":[{clause_no, text}]}`， 识别与结构化在同一次调用中完成；并要求"看不清的字用□占位，不要猜"，避免模型自行补全。

实测：一张 834×542 的中文合同图片，正确识别出 3 条条款，标点完整； 随后走完整审查链路，3.7 秒得到 3 条结论（含一条高风险：缺少租赁期限与地址）。

> 图片合同没有 bbox，因此前端不做 PDF 渲染，只提供条款视图。 这一点在界面上如实标注，不做"看起来和 PDF 一样"的掩饰。

4.8 提示词设计与版本管理

判定提示词由三段构成：审查项定义（名称、风险规则、参考法条）、候选条款原文（每条截断 400 字）、 检索到的法条（top 5，每条截断 300 字），总长度控制在 3000 token 以内。 输出契约固定为 8 个字段的 JSON，并对 `risk_level` 做枚举校验。

提示词版本（`prompt_version`）与清单版本（`checklist_version`）随每条审查记录落库。 改了提示词却没升版本，历史结论就不可复现——这是评审最容易追问的一点。

系统提示词的硬性约束（prompt_version = judge-1.1，摘录）

```
1. 只输出一个 JSON 对象，不要 markdown 代码块，不要任何解释文字。
2. 只依据"合同条款"和"法条依据"判断，不得引用未提供的法条，不得编造法条内容。
3. legal_basis 里的 chunk_id 必须来自提供的法条列表；
   如果法条列表为空，legal_basis 返回 []，且 confidence 不得高于 0.5。
4. 不得给出"胜诉率""一定有效""必然败诉"等确定性表述。
5. suggestion 只写"建议方向 + 常见做法"，不得输出可直接粘贴进合同的完整条文，
   不得使用"甲方应……乙方应……"这类正式条款句式。
6. 输出语言为简体中文。
```

第 3 条与检索侧的空结果提示是同一个约束的两端：模型拿不到法条时，既不能编， 也不能装作很确定——confidence 上限 0.5 会让该条结论自动折叠为"待复核"。

五、核心功能实现

5.1 上传与解析

上传接口先按文件头判断真实类型（PDF / JPEG / PNG），不信任扩展名；文件名一律替换为 UUID， 原始文件名只用于展示。读取时分块累计大小，超限立即中断，避免把大文件读进内存。

解析在请求内同步执行，硬超时 10 秒；超时返回 `2003` 与 `status=parsing`， 任务通过 `asyncio.shield` 继续在后台跑，前端轮询即可拿到结果。 这里没有引入任务队列，代价是必须单进程部署（见 6.3）。

5.2 流式审查

`POST /documents/{id}/review` 返回 `text/event-stream`， 事件类型为 step / finding / progress / done / error。前端用 `fetch` + `ReadableStream` 按空行切帧解析：

```
event: step
data: {"step":"gate","status":"done","message":"从 16 条条款中匹配到 10 处需要核对"}

event: finding
data: {"item_id":"L01","risk_level":"high","clause_no":"第三条","title":"押金退还无时限"}

event: progress
data: {"done":4,"total":10}
```

服务端关闭反向代理缓冲（`X-Accel-Buffering: no`），保证逐条吐出。 实测 19 个事件在 9.8 秒内分批到达（1s / 4s / 7s / 9s），而非结束时一次性输出。

5.3 结论定位

结论与原文的对应关系是这一产品最容易做假的地方。我们的做法是两级：

1. 模型不返回条款 ID，我们从它自己的表述里反推——理由中提到"第X条"就去候选里匹配同号条款；
2. 没有点名的，按文字重合度选候选；仍不明确则取预筛得分最高者，但排除无条号的前置说明段落。

落库时把工作流内部的 `c{seq}` 映射为数据库 `clauses.id`， 前端点击结论即可滚动到对应条款；反过来点击条款也会高亮结论卡片。

5.4 法规资料库

结论里引用的法条必须能被用户自己查到，否则"可溯源"只是一句话。资料库提供三种访问方式：

- 按法规浏览：五部法、1825 条，按条号整数排序分页；
- 法规内筛选：按关键词过滤当前法规；

- 语义检索：走与审查相同的 BM25 检索，返回最相关的 8 条。

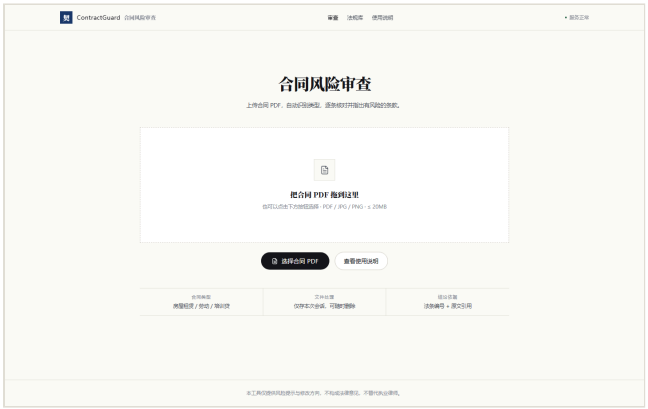
5.5 前端交互

交互	实现要点
三态上传按钮	未选文件 → "选择合同 PDF"；已选 → "开始审查"；上传中 → "解析中"；成功 → "已受理"并停留 420ms 再跳转，避免"点完就没反应"
步骤预告	四步一开始就全部列出（灰点 + 空说明），随事件逐个点亮，判定阶段实时显示 n/m
命中条款导航	审查完成后顶部出现条款按钮，悬停预览等级/依据/置信度，点击跳转
PDF 渐进增强	默认条款视图；后台尝试渲染 PDF，5 秒内成功才出现"原版"切换，失败静默保留文字视图
降级路径	PDF 加载失败、图片合同、无坐标条款，一律回落到条款视图，功能不中断
报告导出	同一份数据渲染两个视图：屏幕上双栏交互，打印时切到单栏报告（`@page` A4、条目不断页）， 走浏览器另存为 PDF。不做服务端模板——报告内容与屏幕完全一致，维护两份样式只会不一致

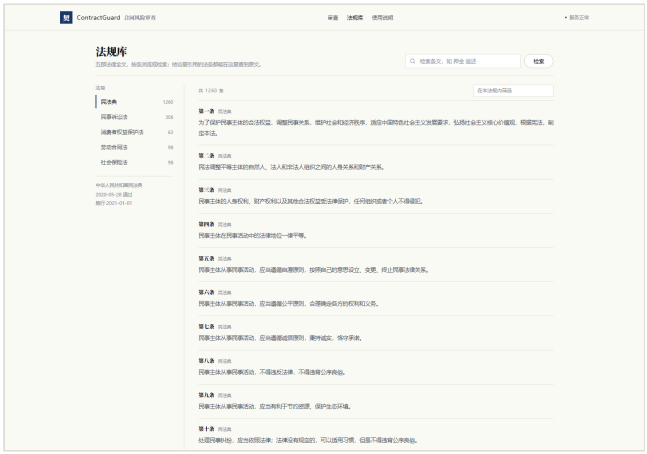
5.6 视觉与文案原则

界面采用"文书感"设计：暖白底、细线分隔、衬线标题、2px 圆角、低饱和风险色。 这套选择的目的不是好看，而是让工具显得可信——法务类产品一旦做成渐变发光风， 用户第一反应是"这是个玩具"。

文案遵循一条规则：只描述用户能做什么，不描述我们多厉害。 因此"可评测""不做开放式审查"这类内部表述只出现在技术文档里，不进入界面。



首页：一份合同、一个入口，投放区占页面主体。



法规库：五部法按条浏览，结论引用的那条能查到原文。

六、测试与部署

6.1 自动化测试

共 84 项测试，覆盖解析、检索、Agent、接口与部署形态、多模态五个层面：

测试文件	数量	覆盖点
test_splitter.py	8	四类编号、无编号兜底、条内款不另起、bbox 并集、偏移单调
test_pdf_parser.py	6	文本行抽取、坐标范围、偏移与全文一致、坏文件报错
test_type_detector.py	8	三类识别、模糊文本判 unknown、用户指定优先、LLM 兜底
test_real_samples.py	6	三份真实示范文本的页数/条数/类型回归，锁定切分结构
test_rag.py	7	语料解析、超长条分段、检索命中、无结果返回空
test_agent.py	10	评分扣分与截断、预筛候选、防幻觉丢弃、兜底标记
test_llm_client.py	11	JSON 清洗、重试、超时、并发闸门（全部注入假 transport，不联网）
test_vision_parser.py	5	图片条款组装、偏移一致、非法输出报错
接口测试	23	上传、详情、原文件、删除、SSE 事件序列、会话隔离、资料库分页与检索、 单进程托管的 SPA 回退（接口 404 不被 HTML 吞掉）

接口测试与多模态测试全部注入假模型，不发真实请求——保证 CI 可离线运行、不消耗额度。

6.2 端到端实测

场景	输入	结果
PDF 解析 + 切分	广东省住房租赁合同示范文本，15 页	514 个文本行 → 15 条条款，类型识别 lease（关键词打分 59 分）
完整审查	同上	10 项清单全部进入判定，6 条中风险，安全分 70，耗时 9.8 秒
图片合同	合同照片 834×542	3 条条款正确识别，审查 3.7 秒，输出 2 条高风险 + 1 条中风险
坏文件	扩展名 .pdf、内容是文本	按文件头拒绝，返回 1002，提示"仅支持真实 PDF"
会话隔离	换 session 查询他人的审查结果	404，不泄露存在性以外信息

6.3 部署

```
cd frontend && npm run build          # 产出 frontend/dist
cd ../backend && python -m scripts.build_index   # 首次：法规索引
uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 1
```

前端构建产物由后端进程直接托管：`/api/v1/*` 走接口，其余路径回落到 `index.html`，写错的接口仍返回 JSON 404 而不是一页 HTML。 这样部署就是"一个进程、一个端口"，演示服务器不必再配 nginx 反代与跨域； 代价是改前端必须重新构建，开发时仍用 vite dev server 走代理。

单进程是硬约束：解析任务、SSE 连接状态、BM25 内存索引都驻留在进程内， 多 worker 会导致任务丢失与索引不一致。启动时会做一次状态修复： 把重启前残留的 `parsing` 记录标记为失败，避免前端无限轮询。

SQLite 在连接时强制执行 `journal_mode=WAL`、`synchronous=NORMAL`、 `foreign_keys=ON`。第三条容易被忽略——SQLite 默认关闭外键， 不显式打开的话 `ON DELETE CASCADE` 完全不生效。

6.4 已知限制

- 评测集尚未完成：现有样本为公开示范文本，条款公平，无法计算"高风险召回率"。真实评测需要带问题的合同并人工标注，计划在省赛前补齐 9 份。
- 法条覆盖仍可加深：已入库五部（民法典、劳动合同法、民事诉讼法、消费者权益保护法、社会保险法）， 但劳动法、个人信息保护法、各地租赁管理条例尚未入库，少数审查项只能引到上位法。
- 图片无坐标：只能做条款级定位。
- 向量通道默认关闭：未配置 embedding 时仅 BM25，同义词召回弱于混合检索。

七、总结与展望

7.1 已完成

从空仓库到可演示的完整链路：合同上传（PDF / 图片）→ 条款切分 → 法规检索 → 逐项判定 → 流式呈现 → 结果落库 → 法规库自查。20 余次提交，84 项测试，三类合同 24 项审查清单， 1358 条法规入库。

7.2 过程中的关键判断

问题	选择与代价
要不要用 LangChain	不用。项目最难的切分、预筛、bbox、防幻觉都不在框架能力范围内，引入只会多一层黑箱；代价是需要自己写检索与编排（约 800 行）
要不要上向量库	不用。语料数千条，numpy 足够；代价是后续扩容需要重写检索层
要不要做 OCR	不做，改用多模态直读；代价是丢掉坐标
要不要做自由审查	不做。固定清单可评测、可复现；代价是覆盖范围受限

7.3 后续计划

1. 补齐评测集：9 份标注合同，输出高风险召回率、准确率、依据可溯率，写入文档与演示视频；
2. 混合检索：接入 embedding，恢复向量通道，改善「押金」这类同义词缺口；
3. 改动对照：原版与修改版并排比对，直观展示"改了什么"；
4. 报告模板：允许用户在打印前选择是否附法条原文，控制报告篇幅。

7.4 反思

这个项目最大的教训是：模型的准确性上限，取决于喂给它的上下文质量。 早期我们花了大量时间调提示词，效果有限；真正带来提升的是三件事—— 把条款切对、把检索 query 改成法条原话、把不相关的条款挡在提示词之外。 换句话说，工程结构决定了 AI 表现的上限，提示词只是在这个上限内做微调。

附录 A：AI 技术选型说明

A.1 模型接入

统一走 OpenAI 兼容协议，通过环境变量切换供应商（`LLM_BASE_URL` / `LLM_MODEL`）， 业务代码不出现任何模型名。封装层负责四件事：超时与重试、全局并发闸门（3）、JSON 契约校验、 失败降级。

A.2 为什么不用 LangChain / LlamaIndex

四条理由，都是针对本项目的具体形态：

1. 最难的模块它不管：中文条款切分、bbox 回填、条款预筛、引用校验，全部要自己写；框架能省的只有 RRF 那几十行。
2. 调试成本高于收益：结论错了要能看到"用了哪个提示词、召回哪几条法条、模型原始输出"，框架的抽象层会挡住这些。
3. 依赖抖动是 9 天窗口里的纯风险：LangChain 主版本迭代频繁，子包版本互相咬合，装出一堆 deprecation 是常态。
4. 工作流形态不匹配：我们是固定四段式流水线，不是动态工具选择；用 Agent 框架跑固定流程，等于给直尺装自动挡。

如果将来要做多轮对话式审查或接入十几个异构数据源，会重新评估——那时框架的价值才成立。

A.3 优化策略

方向	做法	效果
上下文瘦身	条款预筛 + 片段截断（条款 400 字 / 法条 300 字）	单次 prompt < 3000 token
并发	判定 3 并发 + 全局闸门	10 项判定 9.8 秒
稳定性	JSON 失败重试一次；单条失败不拖垮整体	整体成功率显著提升
成本	预筛剔除无关项；小合同只跑 4 项判定	3 条条款的合同 3.7 秒完成

A.4 可解释性

- 结论 → 法条编号 → 原文引用 → 资料库可查，四段可回溯；
- `chunk_id` 在检索结果集合内校验，编造的引用直接丢弃；
- 置信度低于 0.6 折叠为"待复核"，并明确说明原因；
- 提示词版本、清单版本、模型名随结果落库。

附录 B：创新点说明

B.1 技术创新

1. 以"条"为最小语义单位的切分策略：不按固定长度、也不按款级编号切， 而是先判断全文是否存在"第X条"，据此决定切分粒度。解决了条款被切碎后上下文丢失、 高亮框只覆盖一小段的问题。
2. 带引用校验的判定链路：模型返回的每个 `chunk_id` 都要与本次检索结果比对， 不在集合内的引用直接丢弃；无依据时强制压低置信度。把"防幻觉"从提示词层面的嘱咐， 变成代码层面的硬约束。
3. 条款预筛作为成本与准确率的共同杠杆：用关键词 + 文档内 BM25 把笛卡尔积收敛， 既控制了 token 与耗时，也减少了"把无关条款硬套到审查项上"造成的误判。

B.2 应用创新

1. 把法律检索产品化，而不是把大模型套壳：结论必须带法条编号与原文引用， 用户能在内置法规库里自行核对。工具的价值在于"可验证"，而不是"看起来很懂"。
2. 面向三类高频、低门槛合同的收敛式设计：放弃通用性，换取每一类的清单可穷举、 结论可评测。24 项审查覆盖了押金、违约金、解除权、试用期、竞业、退费六个真实痛点。
3. 按输入形态分流的多模态链路：PDF 走文本层抽取（有坐标、可高亮）， 图片走多模态直读（无坐标、条款级定位），两条路复归同一套审查逻辑。 既覆盖了拍照合同这一真实场景，也没有为 OCR 付出安装与体积代价。

B.3 局限的自我说明

我们不把"AI 自动审合同"当作成熟能力来宣传。当前系统能稳定做到的是： 在固定清单范围内，指出可能存在的风险、给出对应法条、提供修改方向。 它不能替代律师，也不应被当作签署依据。把边界写清楚，比扩大宣称更重要。